TrustHome PH

Development, Build & Deployment Guide

Setting up a workstation, the quality gates, Git and coding standards, and the exact runbooks to deploy rules and the frontend safely.

Document	TH-05 · Development, Build & Deployment Guide
Version / date	Rev. 2.0 · 4 October 2026
Factual baseline	TrustHome_Current_Progress_Report_2026-10-04.md (PR #42 `fixes` working copy, based on `features/v1-3-navigation-settings`; excludes `dev`)
Replaces	README Getting Started + Firebase Setup §8, §12
Audience	Team Idle developers, Scrum Master, QA, panel/adviser reviewers, and AI coding assistants used by the team

Evidence label	Meaning (never use these interchangeably)
[Implemented in source]	Code exists in the repository working copy. Not proof that it runs in production.
[Unit/emulator tested]	Covered by automated unit tests or the Firestore emulator suite.
[Reported working]	A team member observed it working in the browser against the test project.
[Deployed/live verified]	Deployed to trusthome-ph and confirmed working there.
[Known gap]	Does not yet meet the product claim. Do not present it as working.
[Roadmap] / [Proposed]	Planned future work / a recommendation in this document, not yet built.

Contents

1. Prerequisites . . . 3
2. Local Setup . . . 3
2.1 Web app . . . 3
2.2 Python scraper and jobs . . . 3
3. NPM Scripts and Quality Gates . . . 4
4. Working with the Firestore Emulator . . . 4
5. Git Workflow . . . 4
5.1 Branches . . . 4
5.2 Commits and pull requests . . . 5
6. Coding Standards . . . 5
7. Deployment Runbooks . . . 6
7.1 Rules and indexes . . . 6
7.2 Hosting (frontend) . . . 6
7.3 Functions (only if Blaze is enabled) . . . 6
8. Release Gate and Smoke Test . . . 6
9. Troubleshooting . . . 7

1. Prerequisites

Tool	Version	Used for
Node.js + npm	18 or later (use the current LTS)	React app, unit tests, lint, build, Firebase CLI
Python	3.10+	Bank scraper, jobs, backups, seeding
Firebase CLI	latest `firebase-tools`	Emulator, rules/indexes/hosting deploys
Java (JDK 11+)	required by the Firestore emulator	Rules tests
Git + GitHub access	—	Branches, pull requests, Actions
Accounts	Firebase project member, Cloudinary user (if uploading), test app accounts	Shared environment

2. Local Setup

2.1 Web app

```
git clone <repo-url> team-idle && cd team-idle
npm install            # use "npm ci" in CI or for an exact reproduction of package-lock.json
npm run dev            # Vite dev server (talks to the shared trusthome-ph project!)
```

> **CAUTION · You are on the shared database**
> Unless `src/firebase.js` is changed, the dev server reads and writes the live `trusthome-ph` project. Use test accounts, mark test data "[TEST]", and coordinate destructive changes.

2.2 Python scraper and jobs

```
cd bank_scraper
python -m venv .venv
# Windows:      .venv\Scripts\activate
# macOS/Linux:  source .venv/bin/activate
pip install -r requirements.txt

python scraper.py --dry-run metrobank              # parse only, no Firestore writes
python -m unittest discover -s . -p "test_*.py"    # 28 tests in the latest run
```

Real Firestore writes require a trusted credential: set `GOOGLE_APPLICATION_CREDENTIALS` to the JSON key path, or `FIREBASE_SERVICE_ACCOUNT_JSON` to its contents. Never put the key inside a folder that is synced or committed.

3. NPM Scripts and Quality Gates

Command	What it does	Must pass before
`npm run dev`	Start Vite dev server with hot reload	—
`npm test`	Node test runner over `src/lib` and guard logic (48 tests in the previous fresh run)	Every PR
`npm run lint`	ESLint (`eslint.config.js`)	Every PR
`npm run build`	Production bundle in `dist/` (vendor chunks split)	Every PR; every deploy
`npm run preview`	Serve `dist/` locally to smoke-test the production build	Every deploy
Rules emulator suite	`firebase emulators:exec --only firestore --"node --test scripts/firestore-rules-smoke.mjs"` (9 tests)	Any PR touching rules or data writes
Python unittest	`python -m unittest discover -s . -p "test_*.py"` in `bank_scraper/`	Any PR touching `bank_scraper/`

BEST PRACTICE · Automate the gates

- **[Proposed]** Add a GitHub Actions workflow `ci.yml` on every pull request: `npm ci`, `npm test`, `npm run lint`, `npm run build`, the Python tests, and the emulator suite. Protect the release branch so merge requires green CI and one approval.
- Re-run all gates after the final edit — the evidence in the progress report is from the *previous* fresh run.

4. Working with the Firestore Emulator

- Use the emulator for anything that could corrupt shared data: rule experiments, migration rehearsals, concurrency tests.
- Seed the emulator with synthetic fixtures inside tests; never import production exports containing personal data.
- Rules tests should assert both success (`assertSucceeds`) and failure (`assertFails`) for each role: signed-out, renter, owner, other user, suspended user, admin.
- To test the booking race, run two approval transactions in parallel against the emulator and assert exactly one Confirmed outcome (expected to fail today — see TH-03 §4.6).

5. Git Workflow

5.1 Branches

Branch	Purpose	Rules
`main`	What is deployed / defense-ready	Protected; merge only via reviewed PR from `dev` or a release branch; tag each release
`dev`	Integration branch	Protected; green CI required
`features/<version>-<topic>`	Feature work, e.g. `features/v1-3-navigation-settings`	Branch from `dev`; short-lived; rebase or merge `dev` often
`fixes` / `fix/<topic>`	Bug fixes, e.g. PR #42 `fixes`	Small, focused; link the bug or test-case ID
`hotfix/<topic>`	Urgent fix to what is deployed	Branch from `main`, merge back to `main` and `dev`

CAUTION · Current branch reality

The baseline working copy (PR #42 `fixes`) is based on `features/v1-3-navigation-settings` and does **not** include `dev`. Merge or rebase onto `dev` and re-run every gate before calling anything "current".

5.2 Commits and pull requests

- Conventional commit prefixes: `feat:`, `fix:`, `docs:`, `test:`, `refactor:`, `chore:` — e.g. `fix(booking): recheck pending status inside transaction`.
- One concern per PR; keep PRs reviewable (< ~400 changed lines where possible).
- PR template: what changed, why, screenshots for UI, test-case IDs covered (TC-...), rules changed? (Y/N), docs updated? (Y/N), deployment notes.
- Reviewers check: rules impact, private data exposure, status transitions, error/empty/loading states, accessibility.
- Never commit: `serviceAccountKey.json`, `.env*` with secrets, `dist/`, `node_modules/`, `.venv/`, Firestore backups.

BEST PRACTICE · AI-assisted coding (team practice)

- Give the assistant TH-02 and TH-03 as context and name the exact files it may change.
- Review every AI change to rules, booking, dispute, trust-score and upload code line by line; run the tests yourself.
- Ask the assistant for tests first (allow + deny cases), then the implementation.
- Never paste credentials, real user data or identity documents into an AI tool.

6. Coding Standards

- **Components**: functional components with hooks; one component per file; keep pages thin and move logic to `src/lib/`.
- **State of every async view**: loading, empty, error (with Retry), success. No blank screens; lazy routes have an accessible fallback; a top-level error boundary offers Reload.
- **Firestore writes**: through the `lib/` module that owns the collection; use batches/transactions for multi-document changes; `serverTimestamp()` for times.
- **Best-effort side effects** (notifications) must never fail the main action; log the failure.
- **Validation** runs in the client for UX and again in rules for security — keep both in sync and test both.
- **Constants** for statuses, roles, collection names and routes.
- **Python**: type hints, `logging` not `print`, dry-run by default for any script that writes, explicit `--apply` to write, idempotent re-runs.
- **Accessibility** is part of "done" (TH-07 §6).

7. Deployment Runbooks

7.1 Rules and indexes

```
git switch main && git pull                    # deploy only reviewed code
firebase emulators:exec --only firestore -- "node --test scripts/firestore-rules-smoke.mjs"
firebase deploy --only firestore:rules,firestore:indexes --project trusthome-ph
```

7.2 Hosting (frontend)

```
npm ci && npm test && npm run lint && npm run build
npm run preview                              # smoke-test the built bundle
firebase hosting:channel:deploy preview-<topic> --project trusthome-ph --expires 7d   # optional preview URL
firebase deploy --only hosting --project trusthome-ph
```

BEST PRACTICE · Deploy order and parity

- Rules first, then hosting, when the new frontend depends on new rules; frontend first when new rules would break the old frontend — decide per change and write it in the PR.
- Use Hosting **preview channels** for remote testers before replacing the live site.
- After deploy, hard-reload (cache cleared) and run the smoke checklist (§8). Tell remote testers only after the live version is confirmed — the booking calendar and admin Download UI are not confirmed live as of 2026-10-04.
- Roll back Hosting from the console (Release history → Roll back) if the smoke test fails.
- Record each deployment: date, commit SHA, target (rules/hosting/indexes), who, smoke result.

7.3 Functions (only if Blaze is enabled)

```
cd functions && npm ci && cd ..
firebase deploy --only functions --project trusthome-ph
```

8. Release Gate and Smoke Test

#	Check	Evidence to record
1	All automated gates green on the release commit (unit, lint, build, emulator, Python)	CI run link
2	Rules deployed from that commit	Deploy log line
3	Hosting deployed from that commit; hard reload shows new build	Timestamp + screenshot
4	Owner creates listing → admin sees notification → approves → listing public	TC-E2E-001 steps 1–2
5	Renter requests → owner sees request → approves → renter notified	TC-E2E-001 steps 3–4
6	Renter logs deposit ref → marks completed → rates	TC-E2E-001 steps 5–6
7	Dispute raised → admin resolves → profile count / status restore correct	TC-E2E-001 step 8
8	Signed-out and regular-user negative checks (TH-04 §9)	Workbook Security rows
9	Bank catalog loads with recent “last seen”; loan estimate shows disclaimer	Screenshot
10	Known gaps listed in the demo notes	Notes file

9. Troubleshooting

Symptom	Likely cause	Fix
`Missing or insufficient permissions`	Rules not deployed, query lacks a filter the rule requires, extra/misspelled field, suspended profile	Check deployed rules version; match query filters to rule; log `request.resource.data` keys in an emulator test
`The query requires an index`	Composite index missing	Add to `firestore.indexes.json`, deploy indexes
Admin page shows “verifying access” then retry	Profile missing/unreadable, `status` not active, or `role` not admin	Inspect `users/{uid}` in console; fix with an admin account
PDF preview/download fails with `show_original_customer_untrusted`	Cloudinary account-level PDF delivery block	See TH-04 §6; do not make documents public
Change works locally but not for testers	Hosting not redeployed or browser cache	Deploy hosting; hard reload; verify build timestamp
Emulator will not start	Port in use or Java missing	Free the port (`firebase.json` emulator ports) or install a JDK
Scraper writes nothing	Run is `--dry-run`, source returned zero valid records (fails closed), or credential not set	Check logs; see TH-06 §2